Instituto Politécnico Nacional
Escuela Superior de Cómputo
ESCOM

Ingeniería en Inteligencia Artificial

Cómputo paralelo

Práctica 2

Análisis y diseño de programas paralelos utilizando procesos

INTEGRANTES

Bustillos Cruz Jonatan
Delgado Lucero Cristian Isaac
Frem Cortés José Angel
Luna Gonzales Gabriel Alexis

Grupo: 6BV1

Ciudad de México a 24 de febrero del 2026

Resumen

Índice general

1. Introducción **1**

2. Marco teórico **2**
2.1. Tipos de datos en C 2
2.1.1. Tipos de datos en la programación 2
2.1.2. Los tipos de datos en C 3
2.1.3. Tipos de enteros 5
2.1.4. Tipos de coma flotante 6
2.1.5. Caracteres 8
2.1.6. Tipo de dato lógico 9
2.1.7. Arrays 11
2.1.8. Punteros 14
2.1.9. Funciones 17
2.1.10. Estructuras 18
2.2. Procesos en C 21
2.3. Archivos en C 21

3. Desarrollo **22**

4. Conclusiones **23**
4.1. Bustillos Cruz Jonatan 24
4.2. Delgado Lucero Cristian Isaac 25
4.3. Frem Cortés José Angel 26
4.4. Luna Gonzales Gabriel Alexis 27

Referencias **28**

A. Anexo de ejemplo **30**

Índice de figuras

2.1. Programa en lenguaje C que ejemplifica el uso de tipos de datos y variables. Fuente: Elaboración propia. 4
2.2. Salida del programa mostrado en la Figura 2.1. Fuente: Elaboración propia. 4
2.3. Programa en lenguaje C que ejemplifica el uso de los tipos de enteros. Fuente: Elaboración propia. 6
2.4. Salida del programa mostrado en la Figura 2.3. Fuente: Elaboración propia. 6
2.5. Programa en lenguaje C que ejemplifica el uso de los tipos de flotantes. Fuente: Elaboración propia. 7
2.6. Salida del programa mostrado en la Figura 2.5. Fuente: Elaboración propia. 7
2.7. Programa en lenguaje C que ejemplifica el uso del tipo char. Fuente: Elaboración propia. 9
2.8. Salida del programa mostrado en la Figura 2.7. Fuente: Elaboración propia. 9
2.9. Programa en lenguaje C que ejemplifica el uso del tipo lógico. Fuente: Elaboración propia. 11
2.10. Salida del programa mostrado en la Figura 2.9. Fuente: Elaboración propia. 11
2.11. Representación en memoria de un arreglo de 10 elementos. Fuente: [4]. 12
2.12. Declaración de arreglos de diferentes tipos de datos y tamaños. Fuente: [4]. 12
2.13. Programa en lenguaje C que ejemplifica el uso de arreglos. Fuente: Elaboración propia. . 13
2.14. Salida del programa mostrado en la Figura 2.13. Fuente: Elaboración propia. 14
2.15. Programa en lenguaje C que ejemplifica la forma para obtener el valor y la dirección de una variable. Fuente: Elaboración propia con base en [2]. 15
2.16. Salida del programa mostrado en la Figura 2.15. Fuente: Elaboración propia. 15
2.17. Relaciones entre *p y el valor de p (dirección de alfa). Fuente: [2]. 16
2.18. Programa en lenguaje C que ejemplifica el uso de apuntadores. Fuente: Elaboración propia con base en [2]. 16
2.19. Salida del programa mostrado en la Figura 2.18. Fuente: Elaboración propia. 16
2.20. Función principal (main) y llamada a la función (func1). Fuente: [4]. 17
2.21. Llamadas a diferentes funciones. Fuente: [4]. 17
2.22. Programa en lenguaje C que ejemplifica el uso de funciones. Fuente: Elaboración propia. 18
2.23. Salida del programa mostrado en la Figura 2.22. Fuente: Elaboración propia. 18
2.24. Programa en lenguaje C que ejemplifica el uso de funciones. Elaboración propia. 20
2.25. Salida del programa mostrado en la Figura 2.24. Fuente: Elaboración propia. 21

Índice de tablas

2.1. Beneficios del uso de tipos de datos en programación. Elaboración propia con base en [1]. 3
2.2. Tipos de datos enteros. Elaboración propia con base en [2]. 5
2.3. Tipos de datos en coma flotante. Fuente: [2]. 7
2.4. Caracteres secuencias (códigos) de escape. Fuente: [2]. 8

Capítulo 1

Introducción

Capítulo 2

Marco teórico

2.1 Tipos de datos en C

2.1.1 Tipos de datos en la programación

Los tipos de datos en programación constituyen una clasificación fundamental que permite definir y representar los valores que un programa puede manipular. Cada tipo de dato cuenta con propiedades y operaciones específicas, lo que determina la forma en que la información se almacena y se procesa dentro de un sistema.

En este sentido, los tipos de datos son esenciales en la declaración de variables. Una variable es un espacio de memoria identificado mediante un nombre simbólico, el cual almacena un valor asociado a un tipo de dato específico. Esto permite a los desarrolladores organizar y gestionar la información de manera eficiente, contribuyendo a la creación de programas más claros, flexibles y reutilizables.

Por lo tanto, al momento de programar, no basta con declarar una variable; también es necesario definir el tipo de dato que le corresponde según el uso que se le dará dentro del programa. De ahí la importancia de conocer los distintos tipos de datos, ya que su correcta elección impacta directamente en el funcionamiento y la claridad del código [1].

La Tabla 2.1 presenta diversos beneficios asociados al uso adecuado de los tipos de datos en programación, los cuales influyen directamente en la calidad del desarrollo de software. Estos aspectos no solo impactan en la correcta ejecución de los programas, sino también en su comprensión, mantenimiento y eficiencia, convirtiéndose en un elemento fundamental dentro de las buenas prácticas de programación [1].

Beneficio	**Descripción**
Claridad y legibilidad del código	Especificar el tipo de datos de una variable hace que el código sea más claro y fácil de entender, permitiendo identificar qué tipo de información almacena.
Validación de datos	Los tipos de datos permiten detectar errores durante la compilación o ejecución, mejorando la confiabilidad del programa.
Almacenamiento eficiente de memoria	El compilador asigna la cantidad adecuada de memoria según el tipo de dato, optimizando el uso de recursos.
Prevención de errores lógicos	El uso correcto de tipos de datos evita operaciones inválidas, como combinar datos incompatibles.
Optimización del rendimiento	Los compiladores pueden aplicar optimizaciones específicas según el tipo de dato utilizado.
Soporte para operaciones específicas	Cada tipo de dato tiene operaciones particulares, como aritmética para enteros o concatenación para cadenas.
Interfaz con bibliotecas y API	Facilita la interacción con funciones externas que requieren tipos de datos específicos como entrada o salida.
Documentación implícita	Los tipos de datos ayudan a entender el propósito de las variables sin necesidad de explicaciones adicionales.
Facilidad de mantenimiento	Permite comprender y modificar el código de forma más sencilla y segura a lo largo del tiempo.

Tabla 2.1: Beneficios del uso de tipos de datos en programación. Elaboración propia con base en [1].

2.1.2 Los tipos de datos en C

El lenguaje C se caracteriza por tener un conjunto reducido de tipos de datos predefinidos, tratándolos internamente de forma numérica. Sin embargo, compensa esta sencillez con una gran flexibilidad para que el programador construya sus propias estructuras de datos complejas [2].

La declaración de variables en el lenguaje C consiste, en su forma más básica, en especificar primero el tipo de dato y posteriormente el nombre de la variable. En caso de requerir asignar un valor inicial, este se incluye a continuación de la declaración. De manera general, la sintaxis se expresa como:

```
<tipo_de_dato><nombre_de_variable>= <valor_inicial>;
```

Asimismo, es posible declarar varias variables del mismo tipo en una sola línea, separándolas mediante comas, como se muestra a continuación:

```
<tipo_de_dato><nombre_de_variable>= <valor_inicial>;
```

El siguiente código ejemplifica el uso de tipos de datos y la declaración de variables en un programa en lenguaje C. En él, se define una variable edad de tipo entero (int) y una variable nombre como un arreglo de caracteres (char), utilizado para representar texto. Posteriormente, se muestran sus valores en pantalla, lo que permite observar cómo los tipos de datos determinan la forma en que la información se almacena y se manipula dentro del programa, tal como se describió previamente. El código fuente se presenta en la Figura 2.1, mientras que el resultado de su ejecución se muestra en la Figura 2.2.

```c
#include <stdio.h>
#include <stdlib.h>

// TEMA. Introducción a los tipos de datos en programación.

int main() {
    // Ejemplo de variable de tipo int.
    int edad = 25;
    // La variable "edad" es de tipo int, representa un número entero.

    // Ejemplo de variable con un arreglo de tipo char.
    char nombre[20] = "Jose";
    // La variable "nombre" es un arreglo de caracteres, la cual representa una cadena de texto.

    // Mostrar los resultados de las variables.
    printf("Edad: %d", edad);
    printf("\nNombre: %s", nombre);

    return 0;
}
```

Figura 2.1: Programa en lenguaje C que ejemplifica el uso de tipos de datos y variables. Fuente: Elaboración propia.

```
Edad: 25
Nombre: Jose
--------------------------------
Process exited after 0.1985 seconds with return value 0
Presione una tecla para continuar . . .
```

Figura 2.2: Salida del programa mostrado en la Figura 2.1. Fuente: Elaboración propia.

Tipos Primitivos o Básicos

Se denominan tipos de datos primitivos aquellos que el compilador reconoce de forma nativa, constituyendo los elementos fundamentales sobre los cuales se construyen los programas. Estos permiten representar información básica dentro de un sistema, facilitando su almacenamiento y manipulación de manera eficiente.

Los tipos de datos primitivos se clasifican principalmente en tres categorías según la naturaleza de la información que almacenan. Los tipos enteros están diseñados para representar números sin parte decimal e incluyen variantes como `char`, `int`, `short`, `long` y `long long`, así como el tipo `enum` para definir conjuntos de constantes. Por otro lado, los tipos reales permiten manejar valores con parte fraccionaria, como `float`, `double` y `long double`, dependiendo de la precisión requerida. Finalmente, los tipos lógicos se representan tradicionalmente mediante valores enteros, donde el valor `0` indica falso y cualquier valor distinto de cero indica verdadero [3].

Tipos Derivados

A diferencia de los tipos de datos primitivos, los tipos derivados se construyen a partir de la combinación o extensión de estos, lo que permite representar estructuras de información más complejas y manejar la memoria de manera más flexible dentro de un programa.

Entre los principales tipos derivados se encuentran los arreglos, que son colecciones de datos del mismo tipo almacenados de forma contigua en memoria; las estructuras y uniones, que permiten agrupar variables

de distintos tipos bajo un mismo nombre; los punteros, que almacenan direcciones de memoria y facilitan un control más directo sobre los recursos del sistema; y las funciones, que, además de representar bloques de código reutilizables, también se consideran tipos derivados debido a que devuelven valores de un tipo específico [3].

2.1.3 Tipos de enteros

El tipo de dato más común en programación es el entero o `int`, el cual es ideal para manejar valores numéricos. Por lo general, estos se almacenan internamente utilizando 2 bytes (16 bits). En la Tabla 2.2 se detallan los tipos de enteros fundamentales, especificando sus rangos de valores y el tamaño en memoria que ocupan según el equipo utilizado.

Existen modificadores como `unsigned`, `short` e `int` que permiten ajustar el rango de estos números. En cuanto a su escritura, el sistema decimal corresponde a la base 10, como en los valores 100 o 450; para valores sin signo se añade el sufijo `U`, por ejemplo, 40000U. El sistema octal, de base 8, se utiliza frecuentemente en sistemas operativos y sus números comienzan con un 0, como en 0377. Por su parte, el sistema hexadecimal, de base 16, emplea el prefijo `0x` seguido de dígitos o letras de la A a la F, como en 0xFF16.

Cuando los tipos básicos no son suficientes para representar cifras muy grandes, se recurre a los enteros de tipo `long`. Existen variantes de este tipo que requieren 4 bytes (32 bits) de almacenamiento. Algunos ejemplos de declaración son `long medida_milimetros;` y `unsigned long distancia_media;`. Para asegurar que el compilador interprete una constante como un tipo largo, se debe agregar el sufijo `L` al final del número, como en la expresión `long numeros_grandes = 40000L;` [2].

Tipo C	**Rango de valores**
int	$-32,768 \cdots +32,767$
unsigned int	$0 \ldots 65,535$
short int	$-128 \cdots +127$
long	$-2,147,483,648 \ldots 2,147,483,647$
unsigned long	$0 \cdots +4,294,967,295$

Tabla 2.2: Tipos de datos enteros. Elaboración propia con base en [2].

El programa presentado a continuación muestra el uso de diferentes tipos de datos enteros en lenguaje C, incluyendo `int`, `unsigned int`, `short int`, `long` y `unsigned long`. Cada variable es declarada con un valor dentro de su rango permitido y posteriormente se imprime en pantalla utilizando la función `printf`, empleando los especificadores de formato adecuados para cada tipo. En la Figura 2.3 se muestra el código fuente del programa, donde se observan las declaraciones y el uso de cada tipo de dato, mientras que en la Figura 2.4 se presenta el resultado de la ejecución, evidenciando cómo se imprimen correctamente los valores asignados.

tipos_enteros.c

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Declaración de variables de distintos tipos enteros.
    int a = -32000;                 // Entero con signo.
    unsigned int b = 65000;         // Entero sin signo.
    short int c = 120;              // Entero corto.
    long d = 2000000000;            // Entero largo.
    unsigned long e = 4000000000;   // Entero largo sin signo.

    // Impresión de los valores.
    printf("Valor de int a: %d\n", a);
    printf("Valor de unsigned int b: %u\n", b);
    printf("Valor de short int c: %d\n", c);
    printf("Valor de long d: %ld\n", d);
    printf("Valor de unsigned long e: %lu\n", e);

    return 0;
}
```

Figura 2.3: Programa en lenguaje C que ejemplifica el uso de los tipos de enteros. Fuente: Elaboración propia.

```
C:\Users\usuario\Desktop\Ser

Valor de int a: -32000
Valor de unsigned int b: 65000
Valor de short int c: 120
Valor de long d: 2000000000
Valor de unsigned long e: 4000000000

--------------------------------
Process exited after 0.232 seconds with return value 0
Presione una tecla para continuar . . .
```

Figura 2.4: Salida del programa mostrado en la Figura 2.3. Fuente: Elaboración propia.

2.1.4 Tipos de coma flotante

Los tipos de datos de punto flotante permiten representar valores numéricos reales, es decir, aquellos que incluyen una parte decimal. Estos datos son útiles tanto para expresar cantidades con precisión, como 3.14159, como para manejar números de gran magnitud mediante notación científica, por ejemplo, $1{,}85 \times 10^{15}$.

La forma de declarar variables de tipo flotante es similar a la utilizada para variables enteras. A continuación se muestran algunos ejemplos:

```
float valor;        /* declara una variable real */
float valor1, valor2;  /* declara varias variables de coma flotante */
float valor = 99.99;   /* asigna un valor decimal a la variable */
```

El lenguaje C admite diferentes formatos de números de punto flotante, como `float`, `double` y `long double`, los cuales varían en precisión y en el espacio de memoria requerido. Generalmente, el tipo `float` ocupa 4 bytes, `double` utiliza 8 bytes, y `long double` puede requerir mayor cantidad de memoria dependiendo del compilador [2].

Tipo C	Rango de valores	Precisión
float	$3{,}4 \times 10^{-38} \ldots 3{,}4 \times 10^{38}$	7 dígitos
double	$1{,}7 \times 10^{-308} \ldots 1{,}7 \times 10^{308}$	15 dígitos
long double	$3{,}4 \times 10^{-4932} \ldots 1{,}1 \times 10^{4932}$	19 dígitos

Tabla 2.3: Tipos de datos en coma flotante. Fuente: [2].

El programa muestra el uso de los tipos de datos `float`, `double` y `long double` en C, así como una comparación básica de su precisión y tamaño en memoria. En la Figura 2.5 se presenta el código, donde se incluye la directiva `#define __USE_MINGW_ANSI_STDIO 1`, la cual se utiliza en compiladores como MinGW para asegurar que funciones como `printf` manejen correctamente formatos avanzados (por ejemplo, `%Lf` para `long double`). Posteriormente, se declaran variables de cada tipo, se imprimen sus valores y se utiliza el operador `sizeof` para conocer cuántos bytes ocupa cada uno. En la Figura 2.6 se observa el resultado de la ejecución, donde se puede notar que, aunque los valores impresos son similares, los tamaños en memoria son diferentes, siendo `float` el de menor tamaño y `long double` el de mayor, lo que está directamente relacionado con la precisión que cada tipo puede manejar.

tipos_flotantes.c

```c
#define __USE_MINGW_ANSI_STDIO 1
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Variables con distinta precisión.
    float a = 3.14f;
    double b = 3.14;
    long double c = 3.14L;

    // Imprimir valores.
    printf("float: %.2f\n", a);
    printf("double: %.2lf\n", b);        // Para double basta con %f o %lf.
    printf("long double: %.2Lf\n", c);   // L mayúscula es obligatoria en long double.

    // Verificación de tamaño para ver cuánto ocupa cada tipo en la computadora.
    printf("\nTamano de float: %zu bytes\n", sizeof(a));
    printf("Tamano de double: %zu bytes\n", sizeof(b));
    printf("Tamano de long double: %zu bytes\n", sizeof(c));

    return 0;
}
```

Figura 2.5: Programa en lenguaje C que ejemplifica el uso de los tipos de flotantes. Fuente: Elaboración propia.

```
C:\Users\usuario\Desktop\Ser

float: 3.14
double: 3.14
long double: 3.14

Tamano de float: 4 bytes
Tamano de double: 8 bytes
Tamano de long double: 16 bytes

--------------------------------
Process exited after 0.252 seconds with return value 0
Presione una tecla para continuar . . .
```

Figura 2.6: Salida del programa mostrado en la Figura 2.5. Fuente: Elaboración propia.

2.1.5 Caracteres

En el lenguaje C, un carácter se entiende como la unidad mínima dentro de un conjunto de símbolos definido, siendo el estándar ASCII uno de los más utilizados en los sistemas de cómputo. Para representar este tipo de información se utiliza el tipo de dato `char`, el cual también puede formar parte de arreglos para construir cadenas de caracteres. Aunque los caracteres se interpretan visualmente como letras o símbolos, internamente se manejan como valores numéricos; por ejemplo, el carácter `'A'` equivale al valor 65 en código ASCII. Esta representación permite realizar operaciones aritméticas sobre ellos, lo que facilita procesos como la conversión entre letras mayúsculas y minúsculas mediante la suma o resta de 32 unidades.

El tipo `char` con signo generalmente cubre un rango de -128 a 127, mientras que la variante `unsigned char` permite representar valores en el intervalo de 0 a 255, abarcando así todo el conjunto ASCII extendido. Dado que los caracteres forman parte de la familia de los enteros, es posible asignarlos a variables de tipo `int` para efectuar operaciones adicionales. Asimismo, el lenguaje C proporciona secuencias de escape para representar caracteres especiales que no pueden escribirse de forma directa, como `\n` para indicar un salto de línea o `\'` para representar una comilla simple. Estas secuencias se presentan en la Tabla 2.4 [2].

Código de escape	**Significado**	**Código ASCII (Decimal)**
`\n`	nueva línea	10
`\r`	retorno de carro	13
`\t`	tabulación	9
`\v`	tabulación vertical	11
`\a`	alerta (pitido sonoro)	7
`\b`	retroceso de espacio	8
`\f`	avance de página	12
`\\`	barra inclinada inversa	92
`\'`	comilla simple	39
`\"`	doble comilla	34
`\?`	signo de interrogación	63

Tabla 2.4: Caracteres secuencias (códigos) de escape. Fuente: [2].

A continuación se presenta un ejemplo que ilustra el uso del tipo de dato `char` en el lenguaje C, así como su relación con la aritmética de enteros y algunas secuencias de escape comunes. En la Figura 2.7 se muestra el código del programa, donde se declara una variable de tipo carácter y se realiza una operación aritmética simple al sumarle una unidad, lo que permite obtener el siguiente carácter según la tabla ASCII. Además, se imprimen tanto los caracteres como sus valores numéricos equivalentes, evidenciando que internamente se manejan como enteros. También se incluyen ejemplos de secuencias de escape como `\n`, `\t` y el uso de comillas, las cuales permiten dar formato a la salida en pantalla. En la Figura 2.8 se presenta la salida obtenida al ejecutar el programa, donde se observan los cambios en los caracteres, sus valores ASCII y el efecto de las secuencias de escape en el formato del texto.

caracteres.c

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Declaración de variables tipo char.
    char letra = 'A';
    char siguiente;

    // Aritmética con caracteres (se manejan como enteros ASCII).
    siguiente = letra + 1;

    // Impresión básica.
    printf("Letra original: %c\n", letra);
    printf("Siguiente letra: %c\n", siguiente);

    // Mostrar valor ASCII.
    printf("ASCII de %c: %d\n", letra, letra);
    printf("ASCII de %c: %d\n", siguiente, siguiente);

    // Uso de caracteres de escape.
    printf("\nEjemplo de salto de linea\n");
    printf("Ejemplo de tabulacion:\tTexto con tab\n");
    printf("Uso de comillas: \"Hola\"\n");

    return 0;
}
```

Figura 2.7: Programa en lenguaje C que ejemplifica el uso del tipo char. Fuente: Elaboración propia.

```
C:\Users\usuario\Desktop\Se

Letra original: A
Siguiente letra: B
ASCII de A: 65
ASCII de B: 66

Ejemplo de salto de linea
Ejemplo de tabulacion:  Texto con tab
Uso de comillas: "Hola"

--------------------------------
Process exited after 0.1163 seconds with return value 0
Presione una tecla para continuar . . .
```

Figura 2.8: Salida del programa mostrado en la Figura 2.7. Fuente: Elaboración propia.

2.1.6 Tipo de dato lógico

Según [2], los compiladores de C que se ajustan al estándar ANSI no contemplan de manera nativa un tipo de dato lógico con valores explícitos como *true* y *false*. En su lugar, este lenguaje emplea el tipo entero para representar condiciones lógicas dentro de estructuras de control como `if` y `while`.

En este contexto, el valor 0 se interpreta como falso, mientras que cualquier valor distinto de cero se considera verdadero. Esto permite construir expresiones lógicas sin requerir un tipo booleano específico, a diferencia de otros lenguajes que sí lo incorporan directamente. Así, cualquier expresión que evalúe a 0 se asocia con falso, mientras que cualquier otro valor entero representa verdadero.

```c
int bisiesto;
```

```
bisiesto = 1;
int encontrado, bandera;
```

A partir de estas declaraciones, es posible utilizar instrucciones como las siguientes:

```
if (encontrado) ...      /* estructura condicional */
indicador = 0;           /* valor falso */
indicador = suma > 10;   /* 1 si se cumple, 0 en caso contrario */
```

De esta forma, en C cualquier valor diferente de cero indica una condición verdadera, mientras que el valor cero representa una condición falsa.

El lenguaje también permite la creación de tipos definidos por el usuario mediante enumeraciones, donde se asignan valores enteros a identificadores simbólicos. Para representar valores lógicos, es común definir un tipo enumerado como el siguiente:

```
enum Boolean { FALSE, TRUE };
```

Con esta declaración se establece un nuevo tipo denominado `Boolean`, que incluye las constantes `FALSE` y `TRUE` asociadas a valores enteros.

Por lo general, las expresiones lógicas se utilizan en estructuras que controlan el flujo del programa. Aunque no suelen emplearse directamente como datos de entrada, sí pueden mostrarse como parte de la salida. Para ello, se utiliza la función `printf()`, como se muestra a continuación:

```
printf("El valor de encontrado es %d\n", encontrado);
```

Si la variable `encontrado` tiene el valor 0, la salida será:

```
El valor de encontrado es 0
```

Enseguida se presenta un programa que ejemplifica el manejo de valores lógicos en el lenguaje C mediante el uso de variables enteras y un tipo definido con `enum`. En la Figura 2.9 se observa el código, donde se evalúa una condición relacional cuyo resultado se almacena en una variable entera, tomando el valor de 1 si la condición es verdadera o 0 en caso contrario. Asimismo, se emplea un tipo enumerado para representar de manera más clara los valores lógicos, facilitando la lectura del programa. Estas variables son utilizadas dentro de estructuras condicionales para controlar el flujo de ejecución. En la Figura 2.10 se muestra la salida del programa, en la cual se pueden apreciar los valores numéricos asociados a las condiciones evaluadas, así como los mensajes generados dependiendo del resultado de las mismas.

tipo_logico.c

```c
#include <stdio.h>
#include <stdlib.h>

// Definición de un tipo lógico usando enum.
enum Boolean { FALSE, TRUE };

int main() {
    int numero = 15;
    int esMayor;
    enum Boolean encontrado = FALSE;

    // Evaluación lógica usando enteros.
    esMayor = numero > 10;  // 1 si es verdadero, 0 si es falso.

    // Uso en estructura condicional.
    if (esMayor) {
        encontrado = TRUE;
        printf("El numero es mayor que 10\n");
    } else {
        printf("El numero no es mayor que 10\n");
    }

    // Mostrar valores lógicos como enteros.
    printf("Valor de esMayor: %d\n", esMayor);
    printf("Valor de encontrado: %d\n", encontrado);

    // Otro ejemplo con condición directa.
    if (encontrado) {
        printf("Condicion verdadera detectada\n");
    }

    return 0;
}
```

Figura 2.9: Programa en lenguaje C que ejemplifica el uso del tipo lógico. Fuente: Elaboración propia.

```
C:\Users\usuario\Desktop\Ser

El numero es mayor que 10
Valor de esMayor: 1
Valor de encontrado: 1
Condicion verdadera detectada

--------------------------------
Process exited after 0.2251 seconds with return value 0
Presione una tecla para continuar . . .
```

Figura 2.10: Salida del programa mostrado en la Figura 2.9. Fuente: Elaboración propia.

2.1.7 Arrays

Un arreglo se define como una agrupación de elementos del mismo tipo de dato que se almacenan de manera contigua en la memoria. Para distinguir cada componente dentro de esta colección, se utiliza un número de orden denominado índice o subíndice.

Al igual que cualquier otra variable, un arreglo debe declararse antes de ser utilizado en un programa. Para definirlo correctamente, es necesario especificar tres elementos fundamentales: el tipo de dato, que determina la clase de valores que almacenará, como `int`, `char` o `float`; el nombre, que funciona como el identificador único del arreglo; y el tamaño, que indica la cantidad máxima de elementos que la estructura puede contener. La sintaxis general para su declaración se escribe como `tipo nombre[tamaño];`, de manera que, por ejemplo, al declarar `int marks[10];` se reserva espacio para diez números enteros como se puede ver en la Figura 2.11.

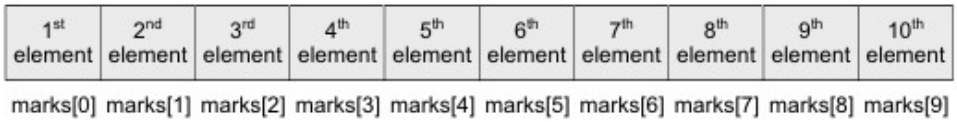

Figura 2.11: Representación en memoria de un arreglo de 10 elementos. Fuente: [4].

Además, para ilustrar cómo se pueden declarar diferentes tipos de arreglos y cómo se representan sus elementos de manera visual, en la Figura 2.12 se muestra un ejemplo gráfico de tres tipos de arreglos en C, cada uno con distintos tipos de datos. La imagen permite observar la estructura y distribución de los elementos dentro de cada arreglo, facilitando la comprensión de cómo se organizan en la memoria del programa [4].

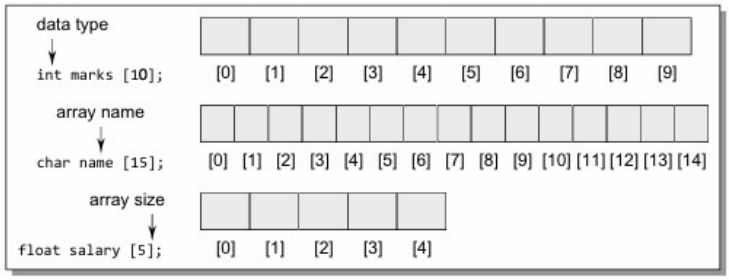

Figura 2.12: Declaración de arreglos de diferentes tipos de datos y tamaños. Fuente: [4].

Posteriormente, se presenta un programa que ejemplifica el uso de arreglos en el lenguaje C con distintos tipos de datos: enteros, flotantes y caracteres. En la Figura 2.13 se muestra el código, donde se declaran e inicializan tres arreglos y se recorren mediante ciclos `for` para acceder a sus elementos. En el caso del arreglo de enteros, se realiza la suma de sus valores, mientras que con el arreglo de flotantes se calcula el promedio de los datos almacenados. Por otra parte, el arreglo de caracteres se utiliza como una cadena de texto, recorriéndose hasta encontrar el carácter nulo que indica el final de la misma. En la Figura 2.14 se presenta la salida del programa, donde se observan los valores almacenados en cada arreglo, así como los resultados de las operaciones realizadas, lo que permite comprender la forma en que se manipulan diferentes tipos de datos mediante estructuras de tipo arreglo.

arrays.c

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Arreglo de enteros.
    int numeros[5] = {10, 20, 30, 40, 50};
    int suma = 0;

    // Arreglo de flotantes.
    float temperaturas[5] = {22.5, 23.0, 21.8, 24.1, 22.9};
    float promedio = 0.0;

    // Arreglo de caracteres (cadena).
    char nombre[] = "Jose";

    // Recorrido de arreglo de enteros y suma.
    int i;
    printf("Arreglo de enteros:\n");
    for(i = 0; i < 5; i++) {
        printf("numeros[%d] = %d\n", i, numeros[i]);
        suma += numeros[i];
    }
    printf("Suma de enteros: %d\n\n", suma);

    // Recorrido de arreglo de flotantes y cálculo de promedio.
    printf("Arreglo de flotantes:\n");
    for(i = 0; i < 5; i++) {
        printf("temperaturas[%d] = %.2f\n", i, temperaturas[i]);
        promedio += temperaturas[i];
    }
    promedio /= 5;
    printf("Promedio de temperaturas: %.2f\n\n", promedio);

    // Recorrido de arreglo de caracteres.
    printf("Arreglo de caracteres (cadena):\n");
    for(i = 0; nombre[i] != '\0'; i++) {
        printf("nombre[%d] = %c\n", i, nombre[i]);
    }

    return 0;
}
```

Figura 2.13: Programa en lenguaje C que ejemplifica el uso de arreglos. Fuente: Elaboración propia.

```
C:\Users\usuario\Desktop\Ser
Arreglo de enteros:
numeros[0] = 10
numeros[1] = 20
numeros[2] = 30
numeros[3] = 40
numeros[4] = 50
Suma de enteros: 150

Arreglo de flotantes:
temperaturas[0] = 22.50
temperaturas[1] = 23.00
temperaturas[2] = 21.80
temperaturas[3] = 24.10
temperaturas[4] = 22.90
Promedio de temperaturas: 22.86

Arreglo de caracteres (cadena):
nombre[0] = J
nombre[1] = o
nombre[2] = s
nombre[3] = e

--------------------------------
Process exited after 0.2599 seconds with return value 0
Presione una tecla para continuar . . .
```

Figura 2.14: Salida del programa mostrado en la Figura 2.13. Fuente: Elaboración propia.

2.1.8 Punteros

Direcciones de memoria

De acuerdo con [2] cuando se declara una variable en C, se le asocian tres características principales: su nombre, su tipo de dato y la dirección de memoria donde se almacena.

El valor de una variable se obtiene utilizando directamente su nombre. Por ejemplo, para mostrar el valor de `n` en pantalla se puede usar:

```
printf("%d", n);
```

Por otro lado, para acceder a la dirección de memoria donde se encuentra almacenada la variable, se utiliza el operador de dirección `&`. Un ejemplo sería:

```
printf("%p", &n);
```

A continuación, en la Figura 2.15, se presenta un programa de ejemplo que permite observar cómo imprimir tanto el valor de una variable como su dirección de memoria. En la Figura 2.16 se muestran los resultados obtenidos. En este caso, el especificador de formato `%d` se utiliza para imprimir el valor entero de la variable, mientras que `%p` permite mostrar la dirección de memoria en formato hexadecimal. Cabe destacar que este valor puede variar dependiendo de la computadora de cada usuario y generalmente se presenta con el prefijo `0x`.

var_dir_mermoria.c

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Se declara una variabla del tipo entero.
    int n = 75;

    // Mostrar el valor de la variable.
    printf("n = %d\n",n);

    // Mostrar el valor de la dirección de memoria de la variable.
    printf("&n = %p\n",&n);

    return 0;
}
```

Figura 2.15: Programa en lenguaje C que ejemplifica la forma para obtener el valor y la dirección de una variable. Fuente: Elaboración propia con base en [2].

```
C:\Users\usuario\Desktop\Se

n = 75
&n = 000000000062FE1C

--------------------------------
Process exited after 0.2335 seconds with return value 0
Presione una tecla para continuar . . .
```

Figura 2.16: Salida del programa mostrado en la Figura 2.15. Fuente: Elaboración propia.

Apuntadores

Al declarar una variable en el lenguaje C, el compilador reserva automáticamente un espacio físico en el sistema para guardar su información, asignando, por ejemplo, dos bytes para un dato de tipo entero. Esta ubicación específica se conoce como dirección de memoria y el sistema accede a ella de forma interna cada vez que el programa requiere consultar o modificar el valor almacenado. No obstante, existe un método para gestionar estas ubicaciones de manera más directa y eficiente mediante el uso de apuntadores.

Un apuntador se define esencialmente como una variable cuyo propósito no es guardar un dato común, sino registrar la dirección de memoria de otra variable. Este concepto es comparable a elementos cotidianos como una dirección postal o un número telefónico, ya que ambos funcionan como indicadores que señalan dónde localizar un destino o una persona específica. En programación, el apuntador actúa como ese mapa que le indica al software exactamente en qué posición de la memoria se encuentran los datos asociados a una variable determinada.

Desde una perspectiva técnica, el apuntador opera bajo la premisa de ser una variable que contiene una dirección apuntando a una posición de memoria ajena, permitiendo que el sistema recupere la información depositada en ese punto exacto. En resumen, cualquier variable que cumpla la función de almacenar una dirección de memoria se denomina formalmente apuntador, sirviendo como un enlace directo hacia la ubicación física de los datos en el hardware [2].

La imagen de la Figura 2.17 ilustra de forma gráfica cómo un apuntador almacena una dirección física de la memoria en lugar de un dato convencional. En el diagrama, se observa que la variable p (ubicada

en la dirección 1000) contiene el valor 100, el cual corresponde precisamente a la dirección donde se encuentra guardada la variable alfa; de esta manera, al utilizar el operador de indirección (*p), el sistema puede acceder directamente al contenido de esa posición, que en este caso es el carácter 'A'.

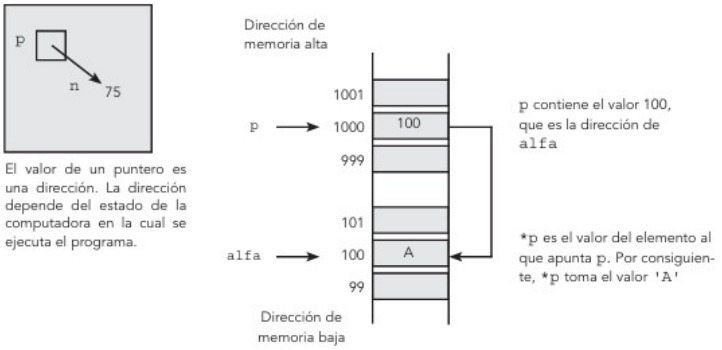

Figura 2.17: Relaciones entre *p y el valor de p (dirección de alfa). Fuente: [2].

El programa en C de la Figura 2.18 pone en práctica el concepto visual al declarar un entero n con el valor 75 y un apuntador p que vincula su dirección mediante el operador &n. A través de las funciones de impresión, en la Figura 2.19, el código demuestra la relación entre la variable original y el apuntador, permitiendo visualizar tanto el valor numérico de n como las direcciones de memoria específicas que el compilador ha asignado para cada variable durante la ejecución.

apuntadores.c

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Variable del tipo entero.
    int n = 75;

    // Variable puntero, la cual tiene la dirección de n.
    int *p = &n;

    // Mostrar los resultados.
    printf("n = %d, &n = %p, p = %p\n", n, &n, p);
    printf("&p = %p\n", &p);

    return 0;
}
```

Figura 2.18: Programa en lenguaje C que ejemplifica el uso de apuntadores. Fuente: Elaboración propia con base en [2].

```
C:\Users\usuario\Desktop\Se

n = 75, &n = 000000000062FE1C, p = 000000000062FE1C
&p = 000000000062FE10

--------------------------------
Process exited after 0.2373 seconds with return value 0
Presione una tecla para continuar . . .
```

Figura 2.19: Salida del programa mostrado en la Figura 2.18. Fuente: Elaboración propia.

2.1.9 Funciones

El lenguaje C permite estructurar los programas en segmentos independientes conocidos como funciones, las cuales facilitan la organización y reutilización del código. Cada función está diseñada para realizar una tarea específica, lo que contribuye a que el programa sea más claro, modular y fácil de mantener. En este sentido, el código de una función se mantiene separado del resto, evitando dependencias innecesarias entre diferentes partes del programa.

Cada función se comunica con el entorno exterior mediante una interfaz definida por su nombre, parámetros y valor de retorno. Esto permite enviar información hacia la función y recibir resultados después de su ejecución. En la Figura 2.20 se ilustra un ejemplo donde la función principal `main()` realiza una llamada a otra función, transfiriendo el control de ejecución. A esta función invocada se le conoce como función llamada, mientras que `main()` actúa como función llamadora. Una vez que la función llamada finaliza su ejecución, el control regresa al punto desde donde fue invocada.

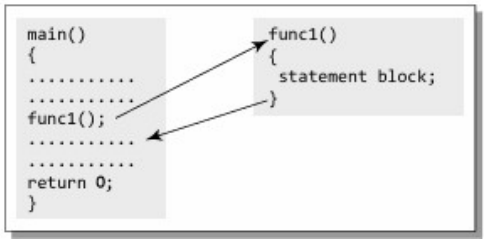

Figura 2.20: Función principal (main) y llamada a la función (func1). Fuente: [4].

Además, una función puede ser utilizada múltiples veces dentro de un programa, incluso dentro de estructuras repetitivas como ciclos `for`, `while` o `do-while`, lo que permite ejecutar la misma tarea varias veces sin necesidad de repetir código. Esto mejora la eficiencia y reduce errores.

No solo la función principal puede invocar otras funciones, sino que cualquier función puede llamar a otra, generando una estructura más compleja de ejecución. En la Figura 2.21 se muestra un ejemplo donde varias funciones se llaman entre sí de manera encadenada, permitiendo dividir el problema en subproblemas más pequeños y manejables [4].

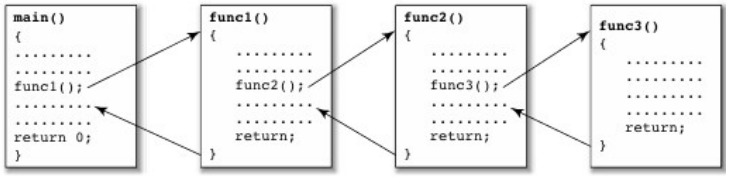

Figura 2.21: Llamadas a diferentes funciones. Fuente: [4].

En esta sección se presenta un ejemplo que ilustra el uso de funciones en el lenguaje C, considerando tanto funciones con valor de retorno como aquellas que no lo poseen. En la Figura 2.22 se muestra el programa, donde se define una función sin retorno que imprime un mensaje en pantalla, así como otra función que recibe parámetros, realiza una operación y devuelve un resultado. Ambas funciones son invocadas desde la función principal `main()`, lo que permite observar la transferencia de control y el uso de valores de retorno. En la Figura 2.23 se presenta la salida del programa, donde se visualiza el mensaje generado por la función sin retorno y el resultado de la operación realizada por la función con retorno.

funciones.c

```c
#include <stdio.h>
#include <stdlib.h>

// Función sin valor de retorno (void).
void saludo() {
    printf("Hola, bienvenido al programa\n");
}

// Función con valor de retorno.
int suma(int a, int b) {
    int resultado = a + b;
    return resultado;
}

// Función principal.
int main() {
    int x = 5, y = 3;
    int resultado;

    // Llamada a función sin retorno.
    saludo();

    // Llamada a función con retorno.
    resultado = suma(x, y);

    // Mostrar resultado.
    printf("La suma de %d y %d es: %d\n", x, y, resultado);

    return 0;
}
```

Figura 2.22: Programa en lenguaje C que ejemplifica el uso de funciones. Fuente: Elaboración propia.

```
C:\Users\usuario\Desktop\Ser

Hola, bienvenido al programa
La suma de 5 y 3 es: 8

--------------------------------
Process exited after 0.2267 seconds with return value 0
Presione una tecla para continuar . . .
```

Figura 2.23: Salida del programa mostrado en la Figura 2.22. Fuente: Elaboración propia.

2.1.10 Estructuras

De acuerdo con [5], una estructura en el lenguaje C es un tipo de dato compuesto que permite agrupar un conjunto finito de elementos que pueden ser de distintos tipos. Este mecanismo resulta útil para representar información más compleja, especialmente cuando se requiere trabajar con datos organizados en forma de registros.

En términos generales, una estructura se define como una colección de variables relacionadas entre sí, pero que pueden pertenecer a diferentes tipos de datos, todas agrupadas bajo un mismo identificador para facilitar su manipulación. Su uso es frecuente en aplicaciones como bases de datos, donde se manejan conjuntos de información organizados en registros.

La sintaxis básica para declarar una estructura es la siguiente:

```
struct tipo_estructura
```

```
{
    tipo_variable nombre_variable1;
    tipo_variable nombre_variable2;
    tipo_variable nombre_variable3;
};
```

En esta definición, `tipo_estructura` corresponde al nombre del nuevo tipo de dato creado, mientras que `tipo_variable` y `nombre_variable` representan los tipos y nombres de los campos que conforman la estructura.

Existen distintas formas de declarar variables de tipo estructura. Una de ellas consiste en definir primero la estructura y posteriormente declarar las variables:

```
struct trabajador
{
    char nombre[20];
    char apellidos[40];
    int edad;
    char puesto[10];
};
struct trabajador fijo, temporal;
```

Otra alternativa es declarar las variables al mismo tiempo que se define la estructura:

```
struct trabajador
{
    char nombre[20];
    char apellidos[40];
    int edad;
    char puesto[10];
} fijo, temporal;
```

En el primer caso, la estructura se define de manera independiente y las variables se crean posteriormente, lo que permite declarar más variables del mismo tipo en otras partes del programa. En cambio, en el segundo caso, las variables se definen junto con la estructura, lo cual limita la posibilidad de declarar nuevas variables de ese tipo más adelante. Es importante señalar que la estructura debe declararse antes de ser utilizada, generalmente antes de la función `main`.

El acceso a los campos de una estructura se realiza mediante el operador punto, utilizando la siguiente sintaxis:

```
variable.campo;
```

Por ejemplo:

```
temporal.edad = 25;
```

En este caso, se asigna el valor 25 al campo `edad` de la variable `temporal`, que pertenece al tipo `trabajador`.

Otra característica relevante es que las estructuras pueden asignarse entre sí, siempre que pertenezcan al mismo tipo:

```
fijo = temporal;
```

Asimismo, es posible inicializar una estructura al momento de su declaración:

```
struct trabajador fijo = {"Pedro","Hernández Suárez", 32, "gerente"};
```

Cuando alguno de los campos es un arreglo, los valores correspondientes deben colocarse entre llaves. Por ejemplo:

```
struct notas
{
    char nombre[30];
    int notas[5];
};
struct notas alumno = {"Carlos Pérez", {8,7,9,6,10}};
```

En esta sección se presenta un ejemplo del uso de estructuras en el lenguaje C para el manejo de registros. En la Figura 2.24 se muestra el programa, donde se define una estructura denominada `Alumno`, la cual agrupa diferentes tipos de datos relacionados, como nombre, edad y promedio. Posteriormente, se declara un arreglo de estructuras que permite almacenar la información de varios alumnos, funcionando como un conjunto de registros. Los datos son capturados mediante entrada estándar y posteriormente mostrados en pantalla utilizando un ciclo. En la Figura 2.25 se presenta la salida del programa, donde se visualiza la información almacenada en cada registro, lo que permite comprender cómo las estructuras facilitan la organización de datos relacionados en un solo elemento.

estructuras.c

```c
#include <stdio.h>
#include <stdlib.h>

// Definición de la estructura.
struct Alumno {
    char nombre[30];
    int edad;
    float promedio;
};

// Función principal.
int main() {
    // Arreglo de estructuras (registros) con datos inicializados.
    struct Alumno grupo[2] = {
        {"Ana", 20, 9.1},
        {"Luis", 22, 8.5}
    };

    // Mostrar la información almacenada.
    printf("Datos de los alumnos:\n");

    int i;
    for(i = 0; i < 2; i++) {
        printf("\nAlumno %d\n", i + 1);
        printf("Nombre: %s\n", grupo[i].nombre);
        printf("Edad: %d\n", grupo[i].edad);
        printf("Promedio: %.2f\n", grupo[i].promedio);
    }

    return 0;
}
```

Figura 2.24: Programa en lenguaje C que ejemplifica el uso de funciones. Elaboración propia.

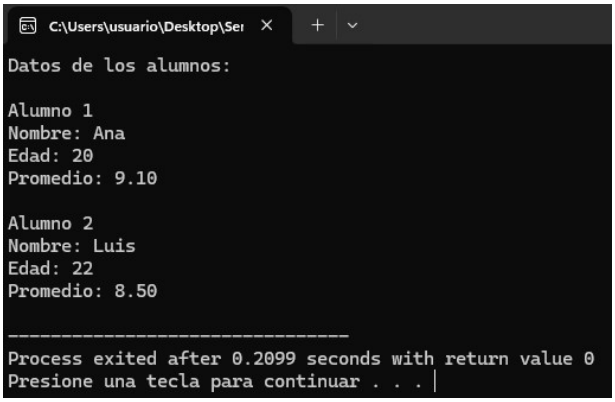

Figura 2.25: Salida del programa mostrado en la Figura 2.24. Fuente: Elaboración propia.

2.2 Procesos en C

2.3 Archivos en C

Capítulo 3

Desarrollo

Capítulo 4

Conclusiones

4.1 Bustillos Cruz Jonatan

4.2 Delgado Lucero Cristian Isaac

4.3 Frem Cortés José Angel

A lo largo de esta práctica realicé un repaso general de los principales conceptos del lenguaje C, enfocándome tanto en los tipos de datos primitivos como en los derivados, lo cual me permitió reforzar conocimientos previamente adquiridos y comprender mejor su aplicación en programas reales. En particular, trabajé con tipos enteros, flotantes, caracteres y lógicos, observando sus características, diferencias de precisión, representación en memoria y formas de uso dentro de estructuras de control y operaciones básicas.

Uno de los aspectos más relevantes fue identificar cómo el lenguaje C maneja internamente ciertos tipos de datos, como el caso de los valores lógicos, que no cuentan con un tipo específico en su forma más básica, sino que se representan mediante enteros. Asimismo, analizar el uso de caracteres y su relación con la tabla ASCII me permitió entender mejor cómo se representan los datos a bajo nivel, al igual que reconocer las diferencias entre `float`, `double` y `long double`, lo cual me ayudó a comprender el impacto de la precisión en los cálculos.

En cuanto a los tipos derivados, desarrollé programas que involucraron arreglos, apuntadores, funciones y estructuras, lo que me permitió entender cómo organizar y manipular datos de manera más eficiente. Por ejemplo, el uso de arreglos me facilitó trabajar con múltiples datos del mismo tipo, mientras que los apuntadores me introdujeron al manejo directo de direcciones de memoria, siendo este uno de los temas que considero más complejos pero también más importantes dentro del lenguaje. Las funciones me permitieron estructurar mejor el código, dividiéndolo en partes más pequeñas y reutilizables, y las estructuras me ayudaron a agrupar diferentes tipos de datos en una sola entidad para representar información más completa.

Durante el desarrollo de los ejercicios, una de las principales dificultades fue mantener atención a los detalles sintácticos del lenguaje, como el uso correcto de operadores, especificadores de formato en `printf` y el manejo adecuado de apuntadores. Estos pequeños errores llegaron a provocar fallos en la ejecución o resultados incorrectos, por lo que me fue necesario ser cuidadoso y revisar constantemente el código. Sin embargo, estas dificultades también me permitieron aprender de los errores y mejorar mi lógica de programación.

Desde un punto de vista práctico, considero que los temas abordados tienen aplicaciones directas en el desarrollo de software, ya que constituyen la base para la construcción de programas más complejos. El correcto uso de tipos de datos, estructuras y funciones me permitió entender mejor cómo se desarrollan programas eficientes, especialmente en áreas relacionadas con el manejo de memoria y la organización de datos.

Además, esta práctica me permitió comprender el análisis y diseño de programas en paralelo utilizando procesos, lo cual amplía la forma en que se pueden desarrollar soluciones, ya que posibilita la ejecución simultánea de tareas. Esto me ayudó a entender la importancia de optimizar tiempos de ejecución y aprovechar mejor los recursos del sistema, así como su relación con el manejo de archivos y la interacción con el sistema operativo.

Como reflexión personal, este repaso me permitió consolidar mis conocimientos y detectar áreas en las que necesito seguir practicando, especialmente en el manejo de apuntadores y en la interpretación de errores. También comprendí que dominar los fundamentos es esencial para avanzar hacia temas más complejos. En general, considero que esta práctica fue útil para reforzar mi lógica de programación y aumentar mi confianza al trabajar con el lenguaje C.

4.4 Luna Gonzales Gabriel Alexis

Referencias

[1] APINEM, *Tipos de datos en programación*, Consultado: 25 de marzo de 2026, s.f. dirección: https://www.apinem.com/tipos-de-datos-programacion/

[2] L. J. Aguilar e I. Z. Martínez, *Programación en C, C++, Java y UML*, 2.ª ed. McGraw-Hill Education, 2014, ISBN: 978-607-15-1212-3.

[3] D. Rivera, *Tipos de datos en C*, Consultado: 25 de marzo de 2026, 2021. dirección: https://blog.pleets.org/article/es/tipos-de-datos-en-c

[4] R. Thareja, *Data Structures Using C*, 2.ª ed. Oxford University Press, 2014, ISBN: 978-0-19-809930-7.

[5] H. Delgado, *Estructuras y Uniones en C - Arrays de estructuras*, Consultado: 27 de marzo de 2026, 2025. dirección: https://disenowebakus.net/estructuras.php

Appendices

Apéndice A

Anexo de ejemplo

Este es un anexo de ejemplo. Puedes agregar aquí información complementaria como diagramas grandes, código extenso, tablas de datos, manuales, etc.